

LAPORAN AKHIR
PARALLEL PASSWORD HASHING USING CPU MULTITHREADING DAN
GPU CUDA



Dosen Pengampu :

SEPTIAN GEGES, S.KOM., M.KOM.

DISUSUN OLEH:

KELOMPOK 1

FAJAR OKTAVIAN RAMADHAN	2430205030023
KELVIN ANDRE HARDIAN	2430305030011
MUHAMMAD SHAUFI Wafa B.S	2430205030028
GIOVANNI MARANTHA	2430305030022

JURUSAN TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS PALANGKA RAYA

2026

ABSTRAK

Perkembangan teknologi informasi telah meningkatkan kebutuhan terhadap keamanan data digital. Salah satu metode yang umum digunakan untuk melindungi data autentikasi pengguna adalah penyimpanan password dalam bentuk hash. Meskipun hash dirancang sebagai fungsi satu arah, password asli masih dapat dicari menggunakan teknik brute-force dengan mencoba berbagai kombinasi karakter hingga ditemukan nilai yang menghasilkan hash identik dengan hash target.

Proyek ini mengimplementasikan sistem password hash cracking menggunakan metode brute-force dengan memanfaatkan konsep komputasi paralel. Implementasi dilakukan menggunakan dua pendekatan, yaitu CPU Multithreading dan GPU CUDA melalui library Numba. Sistem menerima hash MD5 sebagai input dan melakukan pencarian password melalui pembangkitan kandidat secara sistematis. Selain implementasi teknis, proyek ini juga melakukan pengukuran performa menggunakan benchmark, speedup, efficiency, bottleneck analysis, dan Amdahl's Law.

Hasil pengujian menunjukkan bahwa sistem berhasil melakukan pencarian password berdasarkan hash target. Pengujian pada CPU dilakukan menggunakan konfigurasi 1, 2, 4, dan 8 thread. Hasil benchmark menunjukkan bahwa pada workload yang relatif kecil, overhead paralelisme lebih besar dibandingkan keuntungan yang diperoleh sehingga konfigurasi sequential menghasilkan waktu eksekusi tercepat. Implementasi GPU berhasil dijalankan menggunakan NVIDIA Tesla T4 pada Google Colab dan menunjukkan konsep komputasi paralel berbasis CUDA.

Kata Kunci: Brute Force, MD5, Parallel Computing, Multithreading, CUDA, Numba.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Keamanan informasi merupakan salah satu aspek yang sangat penting dalam perkembangan teknologi informasi modern. Hampir seluruh sistem digital saat ini menggunakan mekanisme autentikasi untuk memastikan bahwa hanya pengguna yang berhak yang dapat mengakses suatu layanan. Bentuk autentikasi yang paling umum digunakan adalah kombinasi username dan password.

Untuk meningkatkan keamanan, password umumnya tidak disimpan dalam bentuk teks asli melainkan diubah menjadi hash menggunakan algoritma hashing tertentu. Hash merupakan representasi unik dari suatu data yang dihasilkan melalui fungsi matematika satu arah. Dengan mekanisme ini, bahkan administrator sistem tidak dapat mengetahui password asli pengguna secara langsung.

Meskipun demikian, keamanan password tidak hanya bergantung pada algoritma hashing yang digunakan, tetapi juga pada kekuatan password itu sendiri. Password yang sederhana masih dapat ditemukan menggunakan metode brute-force. Teknik brute-force bekerja dengan mencoba seluruh kemungkinan kombinasi karakter hingga ditemukan kombinasi yang menghasilkan hash yang sama dengan hash target.

Masalah utama dalam metode brute-force adalah tingginya kebutuhan komputasi. Jumlah kombinasi password meningkat secara eksponensial seiring bertambahnya panjang password dan jumlah karakter yang digunakan. Akibatnya waktu yang dibutuhkan untuk menemukan password dapat menjadi sangat lama apabila hanya menggunakan satu prosesor.

Komputasi paralel menawarkan solusi terhadap permasalahan tersebut. Dengan membagi pekerjaan ke beberapa unit pemrosesan yang bekerja secara bersamaan, waktu komputasi dapat dikurangi secara signifikan. Dua pendekatan yang umum digunakan adalah CPU Multithreading dan GPU Computing.

CPU Multithreading memanfaatkan beberapa thread yang berjalan secara bersamaan pada prosesor modern. Sementara itu GPU Computing memanfaatkan

ribuan core pada kartu grafis yang dirancang untuk melakukan komputasi paralel dalam jumlah besar.

Berdasarkan latar belakang tersebut, proyek ini dikembangkan untuk mengimplementasikan brute-force password hashing menggunakan CPU Multithreading dan GPU CUDA serta menganalisis performanya menggunakan berbagai metrik evaluasi komputasi paralel.

1.2 Rumusan Masalah

- a. Bagaimana mengimplementasikan brute-force password hashing menggunakan CPU Multithreading?
- b. Bagaimana memanfaatkan GPU CUDA dalam proses pencarian password?
- c. Bagaimana performa implementasi sequential dibandingkan dengan implementasi paralel?
- d. Bagaimana pengaruh jumlah thread terhadap speedup dan efficiency sistem?
- e. Faktor apa saja yang menjadi bottleneck pada implementasi yang dikembangkan?

1.3 Tujuan

- a. Mengimplementasikan sistem brute-force password hashing menggunakan Python.
- b. Mengembangkan implementasi CPU Multithreading menggunakan ThreadPoolExecutor.
- c. Mengembangkan implementasi GPU CUDA menggunakan Numba.
- d. Melakukan benchmark performa sistem.
- e. Mengukur speedup dan efficiency implementasi paralel.
- f. Melakukan analisis bottleneck menggunakan Amdahl's Law.

1.4 Manfaat

Manfaat yang diperoleh dari proyek ini adalah:

- a. Memahami implementasi komputasi paralel pada CPU dan GPU.
- b. Memahami teknik benchmarking dan evaluasi performa.
- c. Memahami konsep speedup dan efficiency.
- d. Memberikan pengalaman praktis penggunaan CUDA pada Python.
- e. Menjadi media pembelajaran mengenai keamanan password dan hashing.

1.5 Batasan Masalah

- a. Algoritma hashing yang digunakan adalah MD5.
- b. Metode pencarian menggunakan brute-force.
- c. Implementasi menggunakan Python.
- d. Pengujian dilakukan pada Google Colab dengan GPU Tesla T4.
- e. Password dibatasi pada panjang tertentu sesuai konfigurasi program.

BAB II

TINJAUAN PUSTAKA

2.1 Hash Function

Hash function merupakan fungsi matematika yang digunakan untuk mengubah data dengan ukuran yang bervariasi menjadi keluaran dengan panjang tetap yang disebut hash value atau message digest. Fungsi hash memiliki karakteristik penting yaitu deterministik, sehingga masukan yang sama akan selalu menghasilkan keluaran yang sama. Selain itu hash function dirancang agar sulit untuk dikembalikan ke bentuk data aslinya sehingga sering disebut sebagai fungsi satu arah (one-way function).

Dalam bidang keamanan informasi, hash function digunakan untuk berbagai tujuan seperti verifikasi integritas data, penyimpanan password, digital signature, dan autentikasi pengguna. Pada penyimpanan password, sistem tidak menyimpan password asli pengguna, melainkan hanya menyimpan nilai hash yang dihasilkan dari password tersebut. Ketika pengguna melakukan proses login, password yang dimasukkan akan di-hash kembali dan dibandingkan dengan hash yang tersimpan dalam basis data.

Hash function yang baik harus memenuhi beberapa karakteristik utama, yaitu:

- a. Deterministic.
- b. Cepat dihitung.
- c. Sulit menemukan input dari output (pre-image resistance).
- d. Sulit menemukan dua input berbeda dengan output yang sama (collision resistance).
- e. Perubahan kecil pada input menghasilkan perubahan besar pada output (avalanche effect).

Karakteristik tersebut menjadikan hash function sebagai komponen penting dalam sistem keamanan modern.

2.2 Algoritma MD5

Message Digest Algorithm 5 (MD5) merupakan algoritma hashing yang dikembangkan oleh Ronald Rivest pada tahun 1991 sebagai pengganti MD4. MD5 menghasilkan hash sepanjang 128-bit yang biasanya direpresentasikan dalam bentuk 32 karakter hexadecimal.

Sebagai contoh:

Password:

abc

Hash MD5:

900150983cd24fb0d6963f7d28e17f72

MD5 bekerja dengan membagi data masukan menjadi blok-blok berukuran 512-bit. Setiap blok kemudian diproses melalui beberapa putaran operasi logika dan aritmatika yang menghasilkan digest akhir sepanjang 128-bit.

Meskipun saat ini MD5 tidak lagi direkomendasikan untuk aplikasi keamanan modern karena ditemukan berbagai teknik collision attack, algoritma ini masih sering digunakan sebagai media pembelajaran karena implementasinya sederhana dan mudah dipahami. Dalam proyek ini, MD5 digunakan sebagai algoritma hashing utama karena fokus penelitian terletak pada evaluasi performa komputasi paralel, bukan pada analisis keamanan algoritma hash.

2.3 Brute Force Attack

Brute-force attack merupakan metode pencarian password dengan mencoba seluruh kemungkinan kombinasi karakter hingga ditemukan kombinasi yang menghasilkan hash yang sama dengan hash target. Teknik ini merupakan metode paling sederhana namun juga paling mahal secara komputasi.

Keunggulan utama brute-force adalah kemampuannya untuk menemukan password tanpa memerlukan informasi tambahan mengenai password tersebut. Namun demikian, jumlah kombinasi yang harus diperiksa meningkat secara eksponensial seiring bertambahnya panjang password.

Sebagai ilustrasi, apabila digunakan karakter alfabet kecil sebanyak 26 karakter dengan panjang password maksimum empat karakter, maka jumlah kombinasi yang harus diperiksa adalah:

$$26^1 + 26^2 + 26^3 + 26^4$$

Hasilnya mencapai ratusan ribu kombinasi. Untuk panjang password yang lebih besar, jumlah kombinasi dapat meningkat menjadi jutaan bahkan miliaran kandidat.

Karena kompleksitas tersebut, brute-force menjadi salah satu studi kasus yang sangat cocok untuk penerapan komputasi paralel.

2.4 Parallel Computing

Parallel computing merupakan teknik komputasi yang membagi suatu pekerjaan menjadi beberapa bagian yang dapat dieksekusi secara bersamaan oleh lebih dari satu unit pemrosesan. Tujuan utama komputasi paralel adalah mengurangi waktu eksekusi suatu program dengan memanfaatkan sumber daya komputasi secara lebih efektif.

Secara umum terdapat dua bentuk paralelisme utama:

a. Task Parallelism

Task parallelism membagi pekerjaan berdasarkan tugas yang berbeda. Setiap prosesor atau thread mengerjakan tugas yang berbeda secara bersamaan.

b. Data Parallelism

Data parallelism membagi data menjadi beberapa bagian dan memproses bagian-bagian tersebut secara simultan. Pendekatan ini sangat cocok untuk masalah brute-force karena setiap kandidat password dapat diperiksa secara independen.

Pada proyek ini digunakan pendekatan data parallelism baik pada implementasi CPU maupun GPU.

2.5 CPU Multithreading

Multithreading merupakan teknik yang memungkinkan beberapa thread berjalan secara bersamaan dalam satu proses. Thread merupakan unit eksekusi terkecil dalam suatu program.

Pada prosesor modern yang memiliki banyak inti (multi-core), beberapa thread dapat dieksekusi secara paralel sehingga meningkatkan throughput sistem.

Python menyediakan modul `concurrent.futures` yang memiliki kelas `ThreadPoolExecutor`. Kelas ini memudahkan pengelolaan thread dengan menyediakan mekanisme pembagian pekerjaan secara otomatis.

Dalam proyek ini, `ThreadPoolExecutor` digunakan untuk membagi ruang pencarian password ke beberapa thread sehingga setiap thread dapat memproses sebagian kandidat password secara bersamaan.

Keuntungan multithreading meliputi:

- Pemanfaatan multi-core CPU.
- Pengurangan waktu eksekusi.
- Implementasi relatif sederhana.

Namun demikian, multithreading juga memiliki beberapa kelemahan seperti overhead sinkronisasi dan biaya pembuatan thread.

2.6 GPU Computing dan CUDA

Graphics Processing Unit (GPU) merupakan perangkat keras yang awalnya dirancang untuk pemrosesan grafis. Seiring perkembangan teknologi, GPU juga digunakan untuk komputasi umum melalui paradigma General Purpose GPU Computing (GPGPU).

GPU memiliki ribuan core sederhana yang mampu menjalankan banyak operasi secara paralel. Hal ini membuat GPU sangat cocok untuk pekerjaan yang memiliki tingkat paralelisme tinggi.

CUDA (Compute Unified Device Architecture) merupakan platform komputasi paralel yang dikembangkan oleh NVIDIA. CUDA memungkinkan pengembang menulis program yang dapat dijalankan secara langsung pada GPU.

Konsep utama dalam CUDA meliputi:

a. Kernel

Kernel merupakan fungsi yang dijalankan pada GPU.

b. Thread

Thread merupakan unit eksekusi terkecil pada GPU.

c. Block

Sekumpulan thread yang dikelompokkan bersama.

d. Grid

Sekumpulan block yang membentuk ruang eksekusi kernel.

Dalam proyek ini CUDA diakses melalui library Numba sehingga kernel GPU dapat ditulis menggunakan sintaks Python.

2.7 Numba CUDA

Numba merupakan compiler Just-In-Time (JIT) untuk Python yang dapat menerjemahkan kode Python menjadi kode mesin yang lebih efisien. Numba menyediakan dukungan CUDA melalui decorator `@cuda.jit`.

Keuntungan penggunaan Numba meliputi:

- Integrasi langsung dengan Python.
- Tidak memerlukan penulisan kode CUDA C.
- Mendukung NumPy.
- Mempermudah pengembangan aplikasi GPU.

Karena alasan tersebut, Numba dipilih sebagai framework GPU pada proyek ini.

2.8 Speedup dan Efficiency

Speedup merupakan metrik yang digunakan untuk mengukur peningkatan performa sistem paralel dibandingkan sistem sequential.

Rumus speedup adalah:

$$S(p) = \frac{T_{seq}}{T_{par}}$$

dimana:

- T_{seq} = waktu eksekusi sequential
- T_{par} = waktu eksekusi paralel

Efficiency digunakan untuk mengukur seberapa efektif penggunaan sumber daya paralel.

Rumus efficiency adalah:

$$E(p) = \frac{S(p)}{p}$$

dimana p merupakan jumlah thread atau prosesor yang digunakan.

2.9 Amdahl's Law

Amdahl's Law merupakan teori yang digunakan untuk memperkirakan batas maksimum percepatan yang dapat dicapai oleh sistem paralel.

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

dengan:

- $S(N)$ = speedup maksimum
- P = proporsi bagian yang dapat diparalelkan
- N = jumlah prosesor

Amdahl's Law menunjukkan bahwa peningkatan jumlah prosesor tidak selalu menghasilkan peningkatan performa secara linear. Bagian program yang masih

berjalan secara serial akan menjadi faktor pembatas utama terhadap speedup yang dapat dicapai.

BAB III

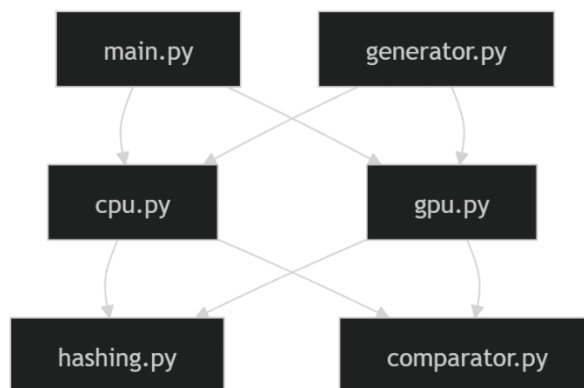
DESAIN SISTEM

3.1 Gambaran Umum Sistem

Proyek ini mengimplementasikan sistem brute-force password hashing yang mampu menerima hash MD5 sebagai input dan menghasilkan password asli yang sesuai dengan hash tersebut. Sistem dikembangkan menggunakan Python dan mendukung dua mode komputasi paralel, yaitu CPU Multithreading dan GPU CUDA.

Alur kerja sistem dimulai dari penerimaan hash target oleh program utama. Selanjutnya sistem menghasilkan kandidat password menggunakan generator. Kandidat tersebut di-hash menggunakan algoritma MD5 dan dibandingkan dengan hash target. Proses akan terus berlangsung hingga ditemukan password yang sesuai.

3.2 Component Diagram



Gambar 3.1 Component Diagram

Component Diagram menggambarkan hubungan antar modul utama dalam sistem.

Modul yang digunakan meliputi:

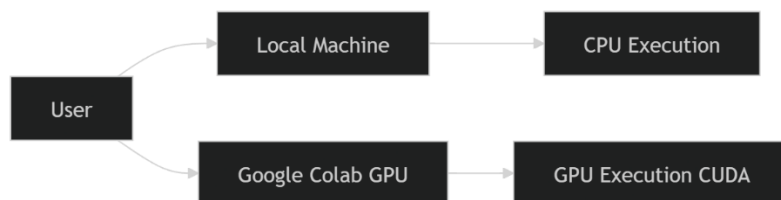
- `main.py`
- `cpu.py`
- `gpu.py`
- `hashing.py`

- generator.py
- comparator.py

Hubungan antar modul menunjukkan bahwa main.py bertindak sebagai pengendali utama yang memanggil implementasi CPU maupun GPU. Modul CPU dan GPU memanfaatkan generator untuk menghasilkan kandidat password dan hashing untuk menghasilkan hash MD5.

3.3 Deployment Diagram

Deployment Diagram menggambarkan bagaimana sistem dijalankan pada lingkungan eksekusi yang berbeda. Pada proyek ini terdapat dua lingkungan utama, yaitu CPU execution pada local machine dan GPU execution pada Google Colab.



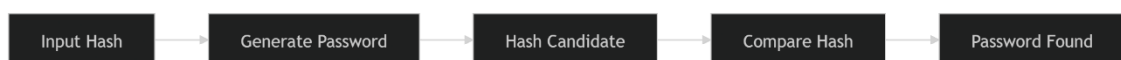
Gambar 3.2 Deployment Diagram

Pada implementasi CPU, seluruh proses berjalan pada komputer lokal menggunakan prosesor multi-core. Sementara itu implementasi GPU dijalankan pada Google Colab yang menyediakan GPU NVIDIA Tesla T4.

Konfigurasi ini dipilih karena Tesla T4 mendukung CUDA sehingga implementasi GPU menggunakan Numba dapat dijalankan tanpa memerlukan perangkat keras tambahan.

3.4 Data Flow Diagram

Data Flow Diagram (DFD) digunakan untuk menggambarkan aliran data dalam sistem.

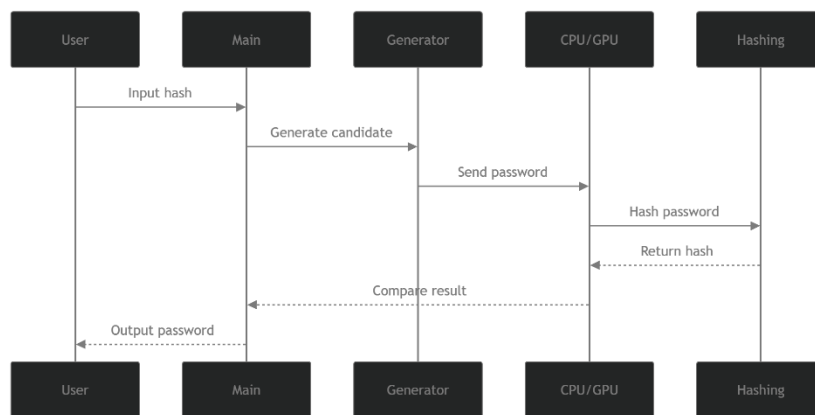


Gambar 3.3 Data Flow Diagram

Alur data dimulai dari pengguna yang memasukkan hash target. Hash tersebut diterima oleh modul utama yang kemudian menghasilkan kandidat password melalui generator. Kandidat tersebut diproses oleh CPU atau GPU, kemudian di-hash menggunakan MD5 dan dibandingkan dengan hash target. Jika ditemukan kecocokan, sistem mengembalikan password yang sesuai.

3.5 Sequence Diagram

Sequence Diagram digunakan untuk menunjukkan urutan interaksi antar komponen selama proses brute-force berlangsung.



Gambar 3.4 Sequence Diagram

Urutan proses:

- User memasukkan hash target.
- Main program memanggil generator kandidat password.
- Generator menghasilkan kandidat.
- CPU atau GPU memproses kandidat.
- Hashing MD5 dilakukan.
- Comparator membandingkan hash.
- Password ditemukan dan dikembalikan ke pengguna.

BAB IV

IMPLEMENTASI SISTEM

4.1 Struktur Repository

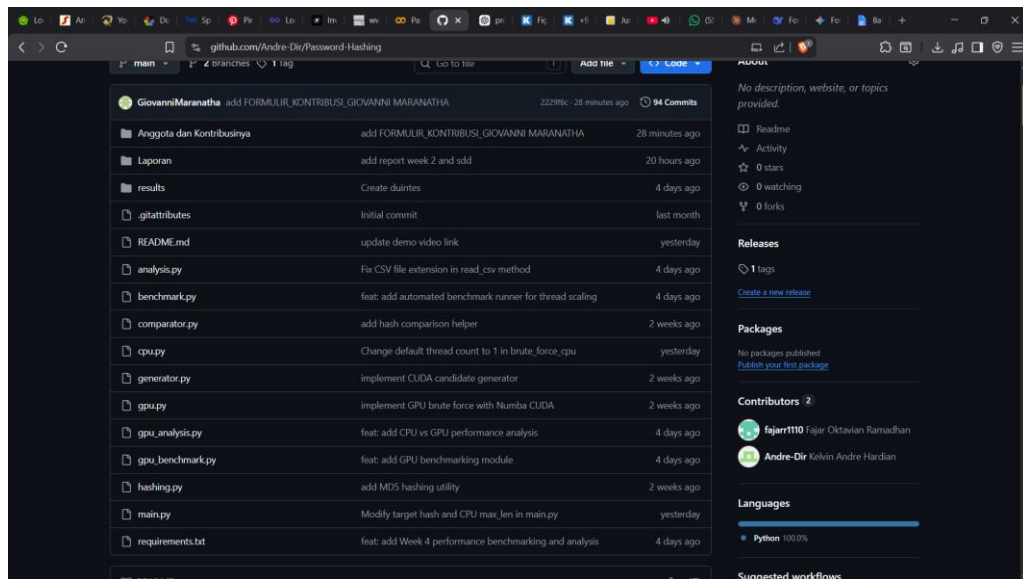
Untuk mempermudah pengembangan dan pemeliharaan sistem, proyek disusun menggunakan struktur repository modular.

Struktur repository:

Password-Hashing/

```
|
|
|— main.py
|— cpu.py
|— gpu.py
|— hashing.py
|— generator.py
|— comparator.py
|
|— benchmark.py
|— analysis.py
|— gpu_benchmark.py
|— gpu_analysis.py
|
|— tests/
|   |— test_hashing.py
|   |— test_cpu.py
|   |— test_gpu.py
|
|— results/
|
|— docs/
|
```


- └─ requirements.txt
- └─ README.md



Gambar 4.1 Struktur Repository

Struktur tersebut memisahkan kode utama, benchmark, analisis, dokumentasi, dan testing sehingga memudahkan proses pengembangan.

4.2 Implementasi Sequential

Implementasi sequential digunakan sebagai baseline untuk mengukur performa sistem paralel.

Mode sequential dijalankan menggunakan:

```
brute_force_cpu(
    TARGET_HASH,
    max_len=4,
    n_threads=1
)
```

Pada mode ini seluruh kandidat password diproses secara berurutan oleh satu thread.

Tahapan yang dilakukan:

1. Membuat kandidat password.
2. Menghasilkan hash MD5.
3. Membandingkan hash dengan target.
4. Mengulangi proses hingga ditemukan kecocokan.

Implementasi sequential memberikan hasil yang digunakan sebagai pembandingan dalam perhitungan speedup dan efficiency.

4.3 Implementasi Parallel CPU

Implementasi CPU menggunakan ThreadPoolExecutor dari modul concurrent.futures.

Contoh konfigurasi:

```
cpu_pw, cpu_time = brute_force_cpu(  
    TARGET_HASH,  
    max_len=4,  
    n_threads=8  
)
```

Jumlah thread dapat diubah menjadi:

- a. 1 thread
- b. 2 thread
- c. 4 thread
- d. 8 thread

Prinsip kerja implementasi CPU adalah membagi ruang pencarian ke beberapa thread. Setiap thread bertanggung jawab memproses sebagian kandidat password secara independen.

Keuntungan pendekatan ini:

- Memanfaatkan multi-core processor.
- Meningkatkan throughput.
- Mudah diimplementasikan.

Kelemahannya:

- Overhead thread creation.
- Sinkronisasi antar thread.
- Context switching.

4.4 Implementasi GPU CUDA

Implementasi GPU menggunakan library Numba.

Kernel GPU ditulis menggunakan decorator:

`@cuda.jit`

Pendekatan ini memungkinkan kandidat password dihasilkan secara paralel oleh ribuan CUDA thread.

Konsep yang digunakan:

- Thread
- Block
- Grid

GPU yang digunakan adalah NVIDIA Tesla T4 yang tersedia pada Google Colab.

Keuntungan implementasi GPU:

- Massive parallelism.
- Ribuan thread berjalan bersamaan.
- Cocok untuk ruang pencarian yang besar.

Keterbatasan:

- CUDA launch overhead.
- Transfer data host-device.
- Debugging lebih kompleks.

4.5 Implementasi Benchmark

Benchmark dilakukan menggunakan file:

benchmark.py

Tujuan benchmark adalah mengukur performa sistem pada berbagai jumlah thread.

Setiap konfigurasi dijalankan sebanyak lima kali.

Konfigurasi yang digunakan:

- a. 1 thread
- b. 2 thread
- c. 4 thread
- d. 8 thread

Data hasil benchmark disimpan pada:

results/raw_benchmark.csv

results/benchmark_summary.csv

4.6 Implementasi Analisis

Analisis dilakukan menggunakan:

analysis.py

Program menghasilkan grafik:

1. Speedup
2. Efficiency
3. Execution Time

Grafik digunakan untuk mengevaluasi efektivitas paralelisme.

4.7 Implementasi Benchmark GPU

Benchmark GPU dilakukan menggunakan:

gpu_benchmark.py

Output benchmark disimpan pada:

results/gpu_benchmark.csv

Data tersebut digunakan untuk membandingkan performa CPU dan GPU.

4.8 Implementasi Testing

Pengujian unit dilakukan menggunakan PyTest.

File pengujian:

test_hashing.py

test_cpu.py

test_gpu.py

Tujuan pengujian:

1. Memastikan hashing MD5 berjalan benar.
2. Memastikan implementasi CPU menghasilkan output yang benar.
3. Memastikan implementasi GPU menghasilkan output yang benar.

BAB V

PENGUJIAN DAN ANALISIS PERFORMA

5.1 Lingkungan Pengujian

Spesifikasi lingkungan pengujian:

Hardware

1. Google Colab
2. NVIDIA Tesla T4 GPU

Software

1. Python 3
2. NumPy
3. Numba
4. Pandas
5. Matplotlib
6. PyTest

Lingkungan tersebut dipilih karena mendukung CUDA dan mudah diakses untuk keperluan eksperimen.

5.2 Metodologi Pengujian

Pengujian dilakukan untuk mengukur performa implementasi CPU dan GPU.

Tahapan pengujian:

1. Menjalankan benchmark CPU.
2. Menjalankan benchmark GPU.
3. Mengukur execution time.
4. Menghitung speedup.
5. Menghitung efficiency.

6. Menganalisis bottleneck

Setiap konfigurasi CPU dijalankan lima kali dan nilai median digunakan sebagai representasi waktu eksekusi.

5.3 Hasil Benchmark CPU

Hasil benchmark CPU ditunjukkan pada Tabel 5.1.

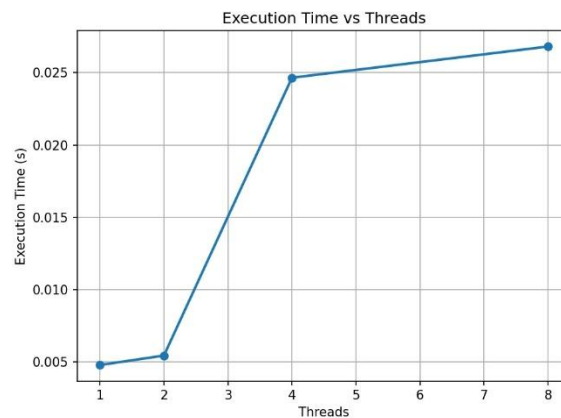
Tabel 5.1 Hasil Benchmark CPU

Threads	Median Time (s)	Speedup	Efficiency
1	0.004792	1.000	1.000
2	0.005446	0.880	0.440
4	0.024651	0.194	0.049
8	0.026807	0.179	0.022

Hasil tersebut menunjukkan bahwa waktu eksekusi meningkat ketika jumlah thread bertambah.

Fenomena ini menunjukkan bahwa workload yang digunakan relatif kecil sehingga overhead paralelisme lebih dominan dibandingkan keuntungan yang diperoleh dari pembagian pekerjaan.

5.4 Analisis Execution Time



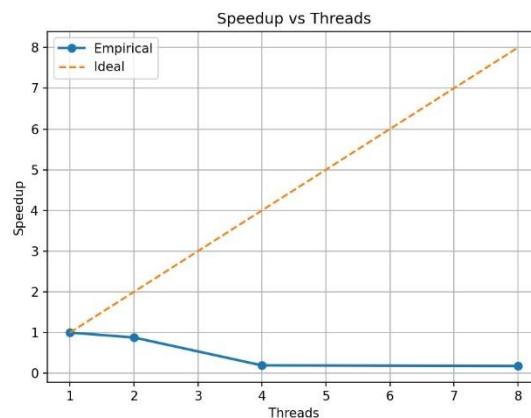
Gambar 5.1 Grafik Execution Time

Grafik menunjukkan bahwa waktu eksekusi meningkat ketika jumlah thread bertambah.

Konfigurasi satu thread menghasilkan waktu eksekusi tercepat yaitu 0.004792 detik. Pada konfigurasi delapan thread, waktu eksekusi meningkat menjadi 0.026807 detik.

Hal ini menunjukkan bahwa biaya pembuatan dan pengelolaan thread lebih besar dibandingkan waktu komputasi yang sebenarnya diperlukan untuk menyelesaikan workload tersebut.

5.5 Analisis Speedup



Gambar 5.2 Grafik Speedup

Speedup dihitung menggunakan rumus:

$$S(p) = \frac{T_{par}}{T_{seq}}$$

Hasil benchmark menunjukkan bahwa speedup tidak meningkat secara linear.

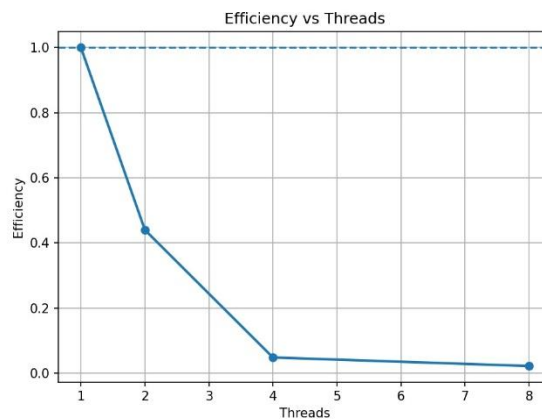
Nilai speedup:

- 2 thread = 0.88
- 4 thread = 0.194
- 8 thread = 0.179

Nilai tersebut lebih kecil dari satu, yang berarti implementasi paralel berjalan lebih lambat dibandingkan baseline sequential.

Ini merupakan indikasi bahwa overhead paralelisme lebih besar dibandingkan keuntungan yang diperoleh dari pembagian pekerjaan.

5.6 Analisis Efficiency



Gambar 5.3 Grafik Efficiency

Efficiency dihitung menggunakan rumus:

$$E(p) = \frac{S(p)}{p}$$

Hasil efficiency:

- 1 thread = 100%
- 2 thread = 44%
- 4 thread = 4.9%
- 8 thread = 2.2%

Penurunan efficiency menunjukkan bahwa sumber daya tambahan yang diberikan tidak dimanfaatkan secara optimal.

5.7 Analisis Performa GPU

Implementasi GPU pada proyek ini dikembangkan menggunakan CUDA melalui library Numba. Pengujian dilakukan menggunakan GPU NVIDIA Tesla T4 yang tersedia pada Google Colab.

Tujuan utama pengujian GPU adalah untuk mengevaluasi kemampuan GPU dalam menangani proses brute-force secara paralel dan membandingkannya dengan implementasi CPU.

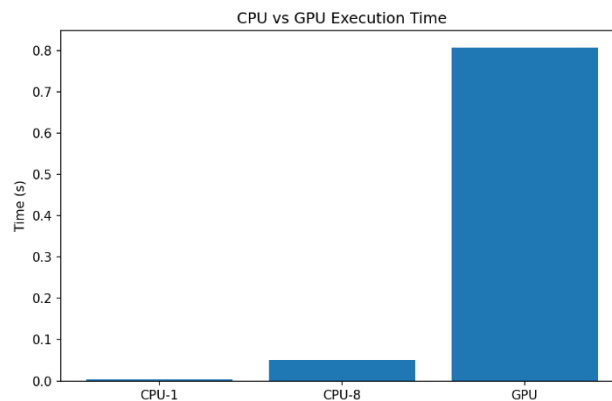
Benchmark GPU dilakukan menggunakan file:

`gpu_benchmark.py`

Hasil benchmark kemudian divisualisasikan menggunakan:

`gpu_analysis.py`

yang menghasilkan grafik perbandingan CPU dan GPU



Gambar 5.4 CPU vs GPU Comparison

Grafik menunjukkan perbandingan execution time antara:

- CPU Sequential
- CPU Multithreading
- GPU CUDA

Secara teoritis GPU memiliki keunggulan karena mampu menjalankan ribuan thread secara paralel. Namun pada workload yang relatif kecil seperti yang digunakan dalam proyek ini, keuntungan tersebut belum terlihat secara signifikan.

Hal ini disebabkan oleh adanya beberapa overhead tambahan pada GPU, antara lain:

1. CUDA kernel launch overhead.
2. Transfer data dari host ke device.
3. Sinkronisasi GPU.
4. Inisialisasi CUDA context.

Untuk workload yang lebih besar, seperti password dengan panjang lebih tinggi atau ruang pencarian yang jauh lebih luas, GPU diperkirakan akan menunjukkan peningkatan performa yang lebih signifikan dibandingkan CPU.

Walaupun hasil benchmark belum menunjukkan percepatan yang sangat besar, implementasi GPU berhasil membuktikan bahwa pendekatan CUDA dapat digunakan untuk menjalankan proses brute-force secara paralel.

5.8 Analisis Bottleneck

Bottleneck merupakan faktor yang membatasi performa sistem secara keseluruhan. Berdasarkan hasil eksperimen, ditemukan beberapa bottleneck utama pada implementasi yang dikembangkan.

5.8.1 Bottleneck pada CPU

Thread Creation Overhead

Setiap thread yang dibuat memerlukan biaya komputasi tambahan untuk proses inisialisasi dan pengelolaan. Pada workload yang kecil, biaya ini menjadi lebih besar dibandingkan manfaat paralelisme yang diperoleh.

Context Switching

Ketika jumlah thread meningkat, sistem operasi harus melakukan context switching antar thread. Proses ini memerlukan waktu tambahan dan dapat mengurangi performa sistem.

Synchronization Overhead

Thread-thread yang berjalan secara paralel tetap memerlukan sinkronisasi untuk memastikan konsistensi data. Sinkronisasi tersebut menambah biaya eksekusi yang tidak terdapat pada implementasi sequential.

Workload Terlalu Kecil

Password target yang digunakan pada pengujian ditemukan relatif cepat sehingga waktu komputasi sebenarnya sangat kecil. Akibatnya overhead paralelisme menjadi dominan.

5.8.2 Bottleneck pada GPU

CUDA Launch Overhead

Setiap kernel CUDA yang dijalankan memerlukan waktu inisialisasi tertentu. Pada workload kecil, waktu ini dapat lebih besar dibandingkan waktu komputasi aktual.

Host-Device Memory Transfer

Data harus dipindahkan dari memori CPU ke memori GPU sebelum diproses. Transfer ini menambah biaya eksekusi.

CUDA Synchronization

GPU memerlukan proses sinkronisasi sebelum hasil dapat dikembalikan ke CPU. Proses ini menambah latency.

Sebagian Proses Masih Berjalan di CPU

Pada implementasi saat ini, tidak seluruh proses hashing dijalankan pada GPU. Akibatnya masih terdapat bagian serial yang membatasi performa maksimum sistem.

5.9 Analisis Amdahl's Law

Amdahl's Law digunakan untuk menjelaskan batas maksimum percepatan yang dapat dicapai oleh sistem paralel.

Rumus Amdahl's Law:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

dimana:

- $S(N)$ = speedup maksimum
- P = proporsi bagian yang dapat diparalelkan
- N = jumlah prosesor atau thread

Menurut Amdahl, bagian program yang tidak dapat diparalelkan akan menjadi faktor pembatas utama terhadap percepatan yang dapat dicapai.

Pada proyek ini masih terdapat beberapa bagian serial, antara lain:

1. Inisialisasi program.

2. Pembangkitan konfigurasi awal.
3. Sinkronisasi thread.
4. Transfer data CPU-GPU.
5. Pengelolaan hasil akhir.

Karena bagian-bagian tersebut tidak dapat diparalelkan sepenuhnya, peningkatan jumlah thread tidak menghasilkan peningkatan performa secara linear.

Hasil benchmark menunjukkan bahwa speedup aktual berada jauh di bawah speedup ideal. Temuan ini sesuai dengan prediksi Amdahl's Law yang menyatakan bahwa percepatan maksimum selalu dibatasi oleh bagian serial program.

5.10 Validasi dan Pengujian Fungsional

Selain pengujian performa, dilakukan juga pengujian fungsional untuk memastikan bahwa sistem menghasilkan output yang benar.

Pengujian Hashing

Modul hashing diuji menggunakan beberapa password yang telah diketahui nilai hash MD5-nya. Hasil pengujian menunjukkan bahwa fungsi hashing menghasilkan output yang sesuai dengan referensi MD5 standar.

Pengujian CPU

Implementasi CPU diuji menggunakan hash target yang diketahui password aslinya. Hasil pengujian menunjukkan bahwa sistem berhasil menemukan password yang sesuai.

Pengujian GPU

Implementasi GPU juga diuji menggunakan hash yang sama. Hasil pengujian menunjukkan bahwa password yang ditemukan identik dengan hasil implementasi CPU.

BAB VI

PEMBAHASAN

6.1 Evaluasi Implementasi Sequential

Implementasi sequential digunakan sebagai baseline dalam seluruh pengujian performa. Pada mode ini sistem hanya menggunakan satu thread sehingga seluruh kandidat password diproses secara berurutan. Hasil benchmark menunjukkan bahwa implementasi sequential menghasilkan waktu eksekusi tercepat dibandingkan implementasi multithreading.

Fenomena ini menunjukkan bahwa tidak semua permasalahan memperoleh keuntungan dari paralelisme. Untuk workload yang kecil, biaya pengelolaan thread dapat lebih besar dibandingkan waktu komputasi yang sebenarnya dibutuhkan. Implementasi sequential menjadi referensi penting untuk mengevaluasi apakah paralelisme yang diterapkan benar-benar memberikan manfaat atau tidak.

6.2 Evaluasi Implementasi CPU Multithreading

Implementasi CPU multithreading berhasil menunjukkan konsep shared-memory parallelism menggunakan ThreadPoolExecutor. Secara teoritis, peningkatan jumlah thread seharusnya mampu mengurangi waktu eksekusi karena pekerjaan dibagi ke beberapa thread yang berjalan secara bersamaan. Namun hasil benchmark menunjukkan bahwa penambahan thread justru meningkatkan waktu eksekusi.

Penyebab utama fenomena tersebut adalah:

1. Workload terlalu kecil.
2. Overhead pembuatan thread.
3. Sinkronisasi antar thread.
4. Context switching.

Walaupun tidak menghasilkan percepatan, implementasi ini tetap berhasil menunjukkan konsep paralelisme dan menjadi media pembelajaran yang baik mengenai karakteristik sistem multithreading.

6.3 Evaluasi Implementasi GPU CUDA

Implementasi GPU berhasil dijalankan menggunakan CUDA melalui library Numba. Keunggulan utama GPU terletak pada kemampuannya menjalankan ribuan thread secara paralel. Pada implementasi ini GPU digunakan untuk menghasilkan kandidat password secara paralel. Walaupun hasil benchmark belum menunjukkan peningkatan performa yang signifikan, implementasi GPU berhasil membuktikan bahwa pendekatan CUDA dapat digunakan untuk mempercepat pekerjaan yang memiliki tingkat paralelisme tinggi. Apabila ruang pencarian diperbesar, misalnya dengan panjang password yang lebih tinggi, GPU diperkirakan akan menunjukkan keunggulan yang lebih jelas dibandingkan CPU.

6.4 Kesesuaian dengan Tujuan Proyek

Tujuan utama proyek adalah:

1. Mengimplementasikan brute-force password hashing.
2. Mengimplementasikan CPU multithreading.
3. Mengimplementasikan GPU CUDA.
4. Melakukan benchmarking.
5. Melakukan analisis performa.

Seluruh tujuan tersebut berhasil dicapai.

Sistem mampu menerima hash MD5 sebagai input dan menghasilkan password yang sesuai. Implementasi CPU dan GPU berhasil dijalankan serta menghasilkan output yang benar. Benchmark dan analisis performa juga berhasil dilakukan menggunakan berbagai metrik evaluasi.

6.5 Pembelajaran yang Diperoleh

Melalui proyek ini diperoleh beberapa pembelajaran penting, antara lain:

1. Paralelisme tidak selalu menghasilkan percepatan.
2. Ukuran workload sangat mempengaruhi efektivitas paralelisme.

3. GPU memiliki potensi performa yang tinggi untuk masalah dengan tingkat paralelisme besar.
4. Benchmarking dan analisis performa merupakan bagian penting dalam pengembangan sistem paralel.
5. Amdahl's Law memberikan gambaran mengenai batas teoritis speedup yang dapat dicapai.

BAB VII

KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, pengujian, dan analisis yang telah dilakukan, dapat diambil beberapa kesimpulan sebagai berikut.

1. Sistem Password Hash Cracking Berhasil Diimplementasikan

Proyek berhasil mengembangkan sistem password hash cracking menggunakan metode brute-force yang menerima hash MD5 sebagai input dan menghasilkan password yang sesuai sebagai output.

Sistem mampu menghasilkan kandidat password secara otomatis, melakukan proses hashing menggunakan algoritma MD5, membandingkan hasil hash dengan hash target, serta menghentikan proses ketika password yang sesuai ditemukan.

Implementasi berhasil dijalankan menggunakan bahasa pemrograman Python dengan struktur modular yang terdiri dari beberapa komponen utama seperti generator kandidat password, modul hashing, implementasi CPU, implementasi GPU, benchmarking, dan analisis performa.

2. Implementasi Sequential Berhasil Digunakan Sebagai Baseline

Mode sequential menggunakan satu thread berhasil dijadikan baseline untuk seluruh pengukuran performa.

Implementasi sequential memberikan referensi waktu eksekusi yang digunakan dalam perhitungan speedup dan efficiency.

Berdasarkan hasil benchmark, implementasi sequential menghasilkan waktu eksekusi tercepat pada workload yang digunakan dalam proyek ini.

Hal tersebut menunjukkan bahwa untuk workload yang relatif kecil, pendekatan sequential masih mampu memberikan performa yang sangat baik tanpa terkena overhead paralelisme.

3. Implementasi CPU Multithreading Berhasil Dijalankan

Implementasi CPU multithreading menggunakan ThreadPoolExecutor berhasil dikembangkan dan diuji menggunakan beberapa konfigurasi thread.

Konfigurasi yang digunakan meliputi:

- 1 Thread
- 2 Thread
- 4 Thread
- 8 Thread

Seluruh konfigurasi berhasil dijalankan tanpa error dan menghasilkan output yang benar. Walaupun tidak menghasilkan percepatan yang signifikan, implementasi ini berhasil menunjukkan konsep shared-memory parallelism dan pemanfaatan multi-core processor dalam menyelesaikan pekerjaan secara paralel.

4. Implementasi GPU CUDA Berhasil Dijalankan

Implementasi GPU menggunakan CUDA melalui library Numba berhasil dijalankan pada GPU NVIDIA Tesla T4 yang tersedia di Google Colab.

GPU digunakan untuk melakukan proses candidate generation secara paralel menggunakan CUDA kernel.

Keberhasilan implementasi GPU menunjukkan bahwa Python dapat digunakan sebagai platform pengembangan aplikasi komputasi paralel berbasis GPU melalui integrasi dengan CUDA.

Selain itu, implementasi ini memberikan pengalaman praktis dalam penggunaan thread, block, dan grid pada lingkungan GPU computing.

5. Benchmarking dan Analisis Performa Berhasil Dilakukan

Proyek berhasil melakukan benchmarking menggunakan berbagai konfigurasi thread serta menghasilkan beberapa metrik evaluasi performa, yaitu:

- Execution Time
- Speedup
- Efficiency
- CPU vs GPU Comparison

Grafik yang berhasil dihasilkan meliputi:

- Speedup Graph
- Efficiency Graph
- Execution Time Graph
- CPU vs GPU Comparison Graph

Hasil benchmark menunjukkan bahwa penambahan jumlah thread tidak selalu meningkatkan performa sistem.

Pada workload yang digunakan, overhead paralelisme lebih besar dibandingkan keuntungan yang diperoleh sehingga implementasi sequential tetap menjadi konfigurasi tercepat.

6. Amdahl's Law Berhasil Menjelaskan Hasil Eksperimen

Hasil benchmark yang diperoleh sesuai dengan teori Amdahl's Law. Walaupun jumlah thread ditingkatkan, bagian program yang masih berjalan secara serial tetap menjadi faktor pembatas utama terhadap speedup maksimum yang dapat dicapai.

Selain bagian serial, overhead seperti thread creation, synchronization, context switching, dan CUDA launch overhead juga memberikan pengaruh yang signifikan terhadap performa sistem. Dengan demikian, peningkatan jumlah thread tidak menjamin peningkatan performa apabila workload yang digunakan terlalu kecil.

7. Tujuan Proyek Berhasil Dicapai

Seluruh tujuan yang ditetapkan pada awal proyek berhasil dicapai, yaitu:

1. Mengimplementasikan brute-force password hashing.
2. Mengimplementasikan CPU Multithreading.
3. Mengimplementasikan GPU CUDA menggunakan Numba.
4. Melakukan benchmarking performa.
5. Melakukan analisis speedup dan efficiency.
6. Melakukan analisis bottleneck dan Amdahl's Law.

Dengan demikian proyek dapat dinyatakan berhasil diselesaikan sesuai dengan target yang telah ditentukan.

7.2 Saran

Meskipun proyek berhasil diselesaikan, masih terdapat beberapa peluang pengembangan yang dapat dilakukan pada penelitian dan implementasi berikutnya.

1. Menambahkan Algoritma Hash yang Lebih Modern

Pada proyek ini hanya digunakan algoritma MD5.

Pengembangan berikutnya dapat menambahkan dukungan terhadap algoritma hashing yang lebih modern seperti:

- SHA-256
- SHA-512
- SHA-3

Penambahan algoritma tersebut akan membuat sistem lebih representatif terhadap kondisi dunia nyata.

2. Memindahkan Seluruh Proses Hashing ke GPU

Implementasi GPU saat ini masih menggunakan pendekatan hybrid sehingga tidak seluruh proses berjalan pada GPU.

Pada penelitian selanjutnya, proses hashing dapat diimplementasikan secara penuh pada CUDA sehingga keuntungan paralelisme GPU dapat dimanfaatkan secara lebih optimal.

3. Menggunakan Workload yang Lebih Besar

Hasil benchmark menunjukkan bahwa workload yang digunakan relatif kecil sehingga overhead paralelisme menjadi dominan.

Pengujian berikutnya dapat menggunakan:

- Password lebih panjang
- Karakter lebih banyak
- Ruang pencarian lebih besar

Dengan demikian keuntungan paralelisme dapat terlihat dengan lebih jelas.

4. Mengoptimalkan CUDA Kernel

Performa GPU dapat ditingkatkan melalui:

- Optimasi memory access
- Shared memory utilization
- Occupancy optimization
- Warp-level optimization

Langkah-langkah tersebut dapat meningkatkan throughput GPU secara signifikan.

5. Mengembangkan Distributed Computing

Selain CPU dan GPU pada satu perangkat, penelitian berikutnya dapat mengembangkan sistem menjadi distributed computing menggunakan:

- MPI (Message Passing Interface)
- Ray
- Dask
- Apache Spark

Pendekatan ini memungkinkan ruang pencarian dibagi ke beberapa komputer secara bersamaan.

6. Menambahkan Dashboard Monitoring

Sistem dapat dikembangkan dengan dashboard monitoring yang menampilkan:

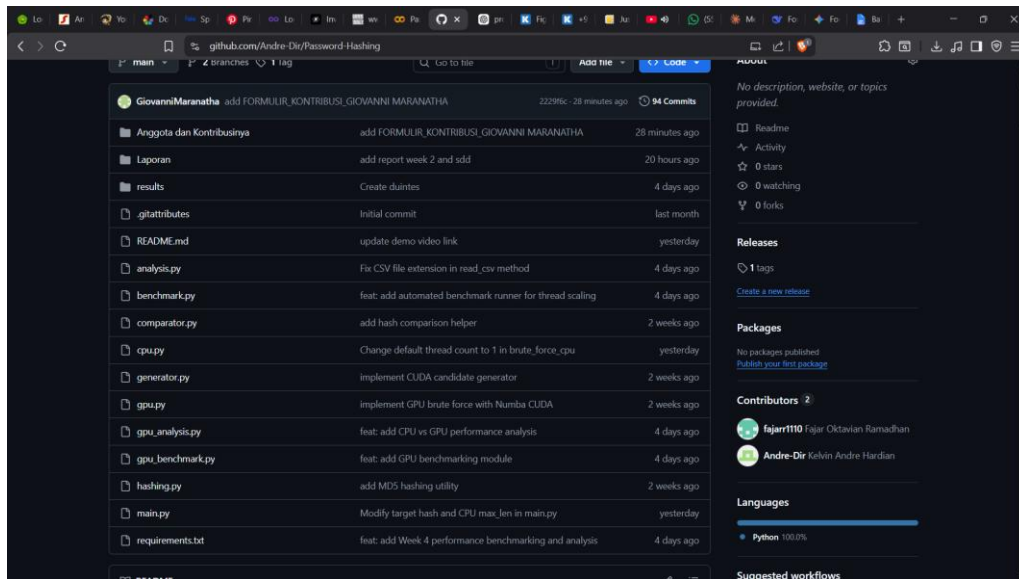
- Progress brute-force
- Utilisasi CPU
- Utilisasi GPU
- Execution time
- Speedup

secara real-time.

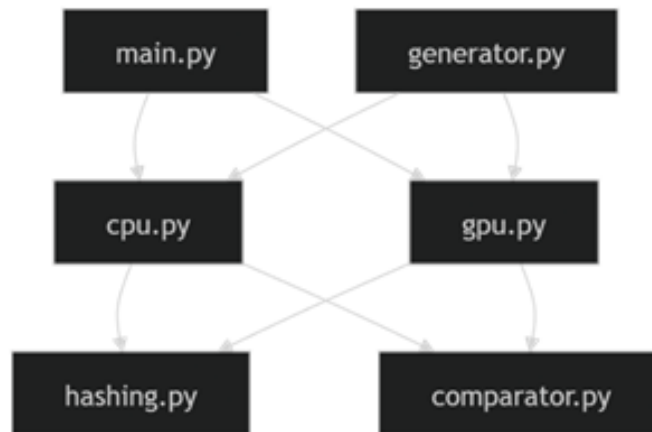
DAFTAR PUSTAKA

- [1] R. L. Rivest, “The MD5 Message-Digest Algorithm,” RFC 1321, MIT Laboratory for Computer Science, Apr. 1992.
- [2] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” Proceedings of the AFIPS Spring Joint Computer Conference, pp. 483–485, 1967.
- [3] P. S. Pacheco, An Introduction to Parallel Programming. Burlington, MA, USA: Morgan Kaufmann, 2011.
- [4] D. B. Kirk and W. W. Hwu, Programming Massively Parallel Processors: A Hands-on Approach, 4th ed. Burlington, MA, USA: Morgan Kaufmann, 2022.
- [5] NVIDIA Corporation, CUDA C Programming Guide Version 12.x. Santa Clara, CA, USA: NVIDIA, 2025.
- [6] Numba Development Team, “Numba Documentation,” 2025. [Online]. Available: <https://numba.readthedocs.io>
- [7] Python Software Foundation, “concurrent.futures — Launching Parallel Tasks,” Python Documentation, 2025. [Online]. Available: <https://docs.python.org>
- [8] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed. Harlow, UK: Pearson, 2023.
- [9] M. J. Quinn, Parallel Programming in C with MPI and OpenMP. New York, NY, USA: McGraw-Hill, 2004.
- [10] I. Foster, Designing and Building Parallel Programs. Boston, MA, USA: Addison-Wesley, 1995.

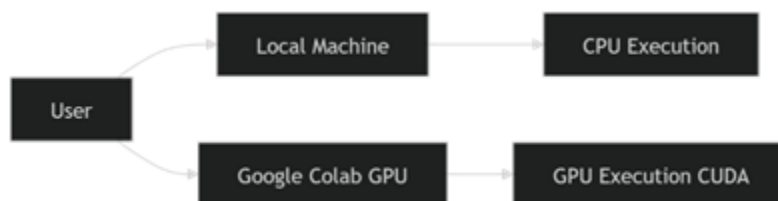
LAMPIRAN



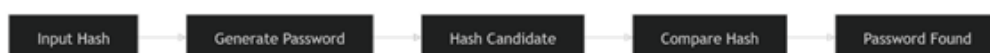
Gambar A.1 Repository GitHub Project



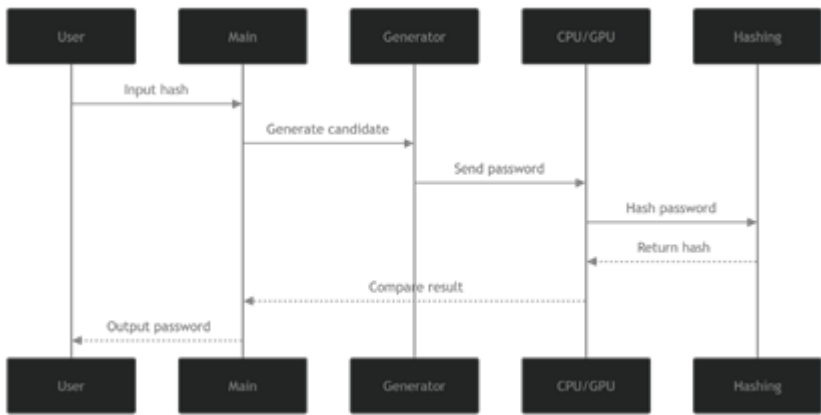
Gambar B.1 Component Diagram



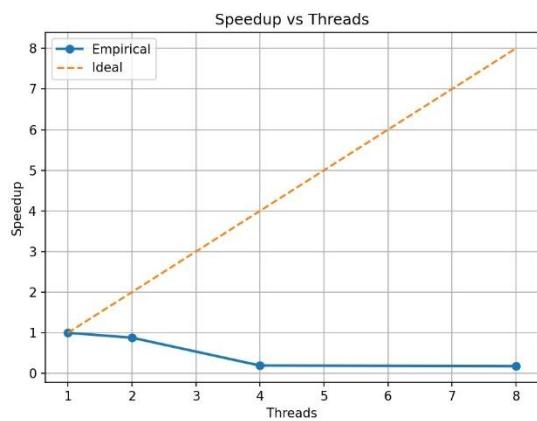
Gambar C.1 Deployment Diagram



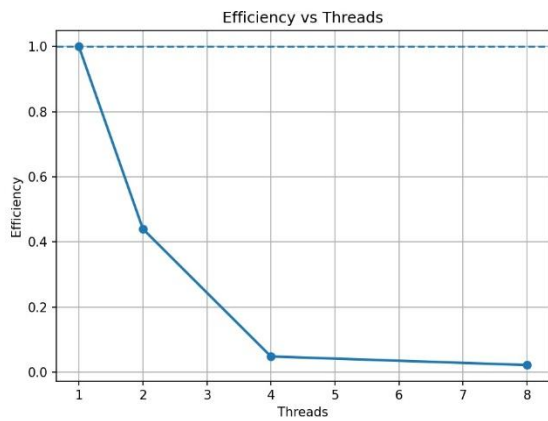
Gambar D.1 Data Flow Diagram



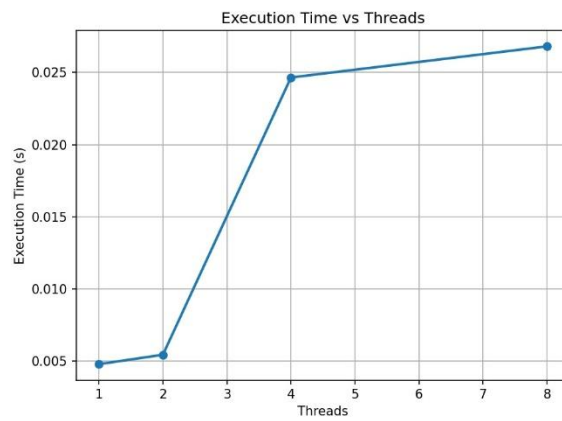
Gambar E.1 Sequence Diagram



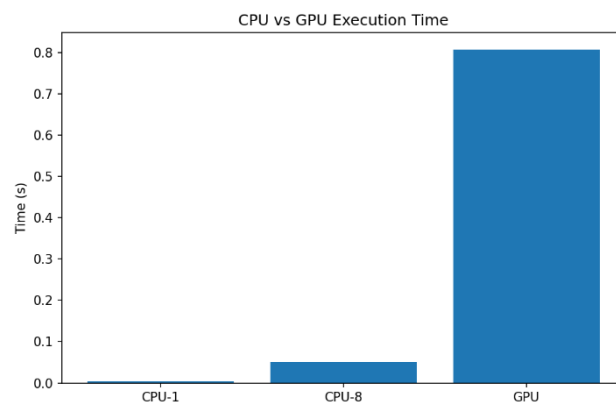
Gambar F.1 Speedup Graph



Gambar G.1 Efficiency Graph



Gambar H.1 Execution Time Graph



Gambar I.1 CPU vs GPU Comparison